VITALO OPERATIONS — ЭТАП 6

MASTER TECHNICAL SPECIFICATION & CLAUDE CODE IMPLEMENTATION PROMPT

Версия: 1.0 · 06 октября 2026

Назначение документа. Это главный технический документ для расширения существующего VITALO MVP в единый Operations / Inventory слой. Документ предназначен для передачи Claude Code. Он не предполагает переписывание существующей системы с нуля.

Ключевой принцип: сначала read-only аудит существующего repository и окружений, затем карта совместимости и план изменений, затем реализация на тестовой среде, и только после проверки — подготовка production deployment.

Статус требований: решения Этапов 1–5 считаются утверждёнными. Если существующий код отличается от описанной архитектуры, Claude не должен молча выбирать вариант. Он должен показать расхождение, сохранить работающую функциональность там, где это возможно, и предложить минимальную миграцию.

1. Контекст существующего VITALO

- VITALO — Telegram-магазин БАДов для Узбекистана. Клиент использует Telegram Mini App; сотрудники работают через web Admin.
- Текущая система построена вокруг единого server API; заявленные технологии: React/Vite/TypeScript для Mini App и Admin, NestJS/TypeScript для API, PostgreSQL/Prisma для БД, Redis/file storage/Docker для вспомогательной инфраструктуры.
- Текущий MVP уже содержит каталог, поиск, корзину, заказ, ручную оплату переводом, резервирование остатков на 12 часов, Admin, роли, журнал действий и Telegram-уведомления.
- Текущий склад quantity-based: фактический остаток, резерв и журнал движений. Индивидуальный Physical ID и рабочий FEFO пока не внедрены.
- Яндекс Маркет пока не является источником данных внутри VITALO. Новая архитектура должна быть source-agnostic: внутренний Order — источник истины; Telegram parser может быть первым адаптером, официальный Yandex API — будущим адаптером.

Источник контекста: предоставленный обзор VITALO от 06.10.2026. В самом обзоре отдельно указано, что он описывает историю разработки и приёмки, а не построчный аудит кода. Поэтому фактическую реализацию Claude обязан проверить в repository.

2. Главная цель Stage 6

- Добавить единый Operations / Inventory слой к существующему VITALO без ненужного переписывания Mini App, Backend и Admin.
- Создать единый физический склад для каналов Yandex Market и VITALO.
- Перейти от quantity-only учёта к учёту каждой физической единицы товара с уникальным Physical ID.
- Ввести FEFO и точное назначение Physical ID при продаже/сборке.
- Добавить закупки Ozon/поставщиков, приёмку, сроки годности, фактическую себестоимость, возвраты, списания, корректировки и журнал движений.
- Объединить Yandex и VITALO в единый Order model, не делая Telegram источником истины.
- Сохранить текущий VITALO pre-payment quantity reserve на 12 часов. Physical ID для VITALO назначается после подтверждения оплаты.
- Сделать Admin реальным рабочим местом; Telegram используется прежде всего как канал уведомлений.
- Сделать warehouse UX простым и task-oriented для жены/родителей: один основной шаг, понятная кнопка, подтверждение, результат.

3. НЕОБХОДИМОЕ ПРАВИЛО: AUDIT FIRST

Перед любыми изменениями Claude обязан выполнить read-only аудит. На этой фазе запрещено делать миграции, массовые изменения данных, менять production configuration или переписывать код.

- Определить фактическую структуру repository: приложения, packages, services, shared modules, scripts, environment files.
- Найти фактические модели Prisma и таблицы: Product, Order, OrderItem, stock/reservation, User/Employee/Role, Action Log и связанные сущности.
- Проследить текущий flow заказа от Mini App до API, БД, Admin и Telegram notifications.
- Проверить текущую логику 12-часового quantity reserve и момент, когда резерв превращается в списание.
- Проверить существующие transaction boundaries, idempotency и защиту от double-click/double stock movement.

- Проверить конфигурацию local/test и production: DB URL, Redis, storage, bot tokens, API endpoints, Docker/compose, secrets и другие ENV.
- Установить, разделены ли test и production БД. Если нет — остановить любые опасные операции и предложить безопасное разделение.
- Определить способ запуска, тестирования, миграций и rollback.
- Составить Compatibility Matrix: Already Exists / Extend / New / Conflict / Needs Decision.

4. Правила работы с окружениями

- Локальная тестовая среда является основной средой разработки Operations.
- Production нельзя использовать как playground.
- Никакие destructive migration, reset, seed или массовые UPDATE/DELETE не выполнять в production без явного подтверждения.
- Перед изменениями БД Claude должен определить, какая БД подключена текущему процессу.
- До production rollout должны существовать backup и понятный rollback plan.
- Если текущая инфраструктура не соответствует этим правилам, сначала предложить минимальный безопасный способ исправления.

5. Архитектурный принцип совместимости

```
For every existing feature:
1. Inspect actual implementation.
2. Reuse it if it already satisfies the requirement.
3. Extend it if only additional fields/flows are required.
4. Refactor only when necessary for correctness or maintainability.
5. Rewrite only with explicit justification and impact analysis.
```

Нельзя создавать вторую параллельную систему заказов, товаров, пользователей или склада только ради Operations. Новые экраны должны работать поверх единого набора сущностей.

6. Утверждённая структура Admin

- Dashboard — текущее состояние бизнеса.
- Задачи — task-oriented интерфейс для склада/семьи.
- Заказы — единый operational Order list для Yandex + VITALO.
- Продажи — фактические финансовые продажи, не копия Orders.
- Склад — Physical Units, остаток, приёмка, возвраты, списания, корректировки, журнал.
- Закупки — Ozon и поставщики, ожидаемые/полученные/отменённые количества, discrepancies.
- Товары — Product Master и справочники.
- Аналитика — причины и показатели результата.
- Расходы — Admin-only.
- Маркетинг — контент, маркетинговые данные и эффективность.
- Настройки — сотрудники, роли, справочники, печать, Telegram, Action Log.

7. Product / SKU

- Один основной SKU продукта, желательно совпадающий с SKU Yandex Market.
- Product Master: SKU, название RU/UZ, категория, бренд, производитель, страна, variant, form, unit, quantity/weight/volume, composition RU/UZ, description RU/UZ, usage, contraindications, search keywords, photos, Yandex price, VITALO price, Yandex commission %, Yandex logistics fixed.

- Категория у продукта одна. Категории можно создавать/переименовывать/архивировать.
- Справочники Brand, Manufacturer, Country, Form, Unit, ExpenseCategory, Supplier, OzonAccount управляются Admin.
- Не вводить на MVP отдельную систему внешних supplier SKU/EAN/GTIN, если её нет и она не нужна для текущего flow.

8. Physical Unit — центральная сущность склада

- Каждая физическая банка/единица получает уникальный ID, например V-000184.
- Physical ID создаётся автоматически при приёмке.
- Каждая единица имеет собственный expiryDate, даже если UI позволяет вводить одинаковую дату сразу для группы.
- Каждая единица связана с конкретной Purchase/PurchaseLine и фактической себестоимостью.
- Physical ID не удаляется и не переиспользуется.
- Внутренняя термоэтикетка печатается через XPrinter 365B. В ней: Physical ID/штрихкод, SKU, состав RU/UZ, срок годности.
- Yandex order number не печатается на внутренней этикетке банки; официальный Yandex label находится на внешней упаковке.
- Сканирование желательно; ручной ввод всегда остаётся fallback.
- Нет учёта мест хранения/полок на MVP.

9. FEFO и MISSING

```
FEFO selection:
- select available PhysicalUnit for SKU with earliest valid expiry;
- preassign exact Physical ID;
- persist assignment;
- if assigned unit is physically missing:
1) mark unit MISSING;
2) create movement/action log and FEFO exception;
3) do not block the order;
4) propose the next available FEFO unit;
5) require confirmation of replacement.
```

Строгий вариант выбран: система заранее назначает конкретный Physical ID. Нельзя просто записать «1 штука из SKU» и выбирать банку позже.

10. Purchases

- Purchase имеет sourceType: OZON или SUPPLIER.
- Ozon Purchase: Ozon Account, Ozon Order Number, Purchase Date, lines SKU + ordered quantity + original unit price, Final Actual Purchase Price.
- Если Final Actual Purchase Price отличается от суммы line prices, разница распределяется пропорционально по line items; каждая Physical Unit получает effective actual cost.
- Purchase не удаляется. Отмена — отдельное событие.
- Ozon cancellation workflow: ввод Ozon order number → найти Purchase → выбрать cancelled quantity → сохранить Ordered / Cancelled / Remaining/Expected.
- Supplier receiving использует тот же receiving engine, но без обязательной привязки к Ozon account/order.

11. Ozon Receiving

```
Warehouse flow:
Ozon Account
→ Expected positions
→ Select product
→ Enter actual received quantity
→ Enter expiry for each unit/group
→ Generate Physical IDs
→ Print labels
→ Finish
→ Units become IN_STOCK
```

- Родителям/жене не требуется вводить Ozon order number, purchase date или price при обычной приёмке.
- Expected 2 / Received 1 → 1 In Stock + 1 Expected.
- Expected 2 / Received 3 → принять 3 и создать discrepancy.
- Discrepancy не должен уничтожать исходные expected values.

12. Unified Stock Model

- Один физический склад для Yandex и VITALO.
- Нет отдельных channel stock pools.
- Все свободные Physical Units доступны обоим каналам.
- Пример: всего 20; Yandex delivery 3; VITALO delivery 2; свободно 15.
- Назначение единицы каналу не является окончательной продажей.
- Финальное списание происходит только при переходе в YANDEX_SOLD или VITALO_SOLD.

13. Yandex Order Architecture

- Внутренняя сущность Order является source of truth.
- Первый адаптер может получать заказ из сообщений Yandex в Telegram; архитектура должна позволять заменить этот адаптер официальным Yandex API без переписывания Order core.
- Telegram не является источником истины о состоянии заказа.
- Основные внутренние состояния Yandex: Яндекс доставка → Яндекс продано / Яндекс возврат.
- При создании/import Yandex order: SKU → FEFO → exact Physical ID → assignment → assembly task → Yandex delivery.
- Если клиент получил заказ: пользователь вручную проверяет Yandex Market и нажимает «Клиент получил» → Yandex sold.
- Возврат: Yandex delivery → Yandex return → физически вернулся → scan/enter Yandex order number → найти Physical ID → принять на склад → In Stock или Write-off.
- Нельзя принять один Yandex return дважды.

14. VITALO Order — сохранить существующий flow

```
Existing VITALO flow to preserve:
Create order
→ existing 12h quantity reserve
→ customer transfers money
→ customer clicks "Я оплатил(а)"
→ payment checking
→ Manager confirms payment
→ existing reserve becomes deduction / paid
→ physical FEFO assignment
→ assembly
→ delivery
```

```
→ delivered
→ VITALO sold
```

Критически важно: Physical ID не должен назначаться вместо существующего pre-payment quantity reserve. Сначала сохраняем текущий рабочий резерв, после подтверждения оплаты назначаем конкретную физическую единицу.

15. Sales and historical economics

- Sale — отдельная историческая запись факта продажи.
- Sale содержит order, channel, SKU/product, Physical ID, quantity, sale price, costAtSale.
- Для Yandex на момент продажи фиксируются commissionAtSale и logisticsAtSale.
- Изменение будущей комиссии/логистики не пересчитывает исторические продажи.
- Advertising expenses учитываются отдельно и могут распределяться в аналитике по периоду/channel.
- Profit/contribution считается из зафиксированных исторических значений, а не из текущей карточки Product.

16. Returns, Write-off, Corrections

- Returns — отдельный operational section, не смешивать с успешными Sales.
- Write-off — отдельный section. Warehouse может выполнить списание.
- Write-off требует второго подтверждения.
- Комментарий не обязателен; короткая причина может быть предложена.
- Correction доступна Admin/Manager и всегда пишется в Action Log.
- Исторические данные не удаляются. Использовать event/correction/cancellation.

17. Roles

Role	Основной доступ
Admin	Всё, включая finance, settings, employees, full control
Manager	Полный operational access: orders, payments, warehouse, receiving, assembly, returns, write-off, journal; без finance
Warehouse	Receiving, tasks, assembly, returns, write-off, correction в разрешённом объёме; без finance/purchase cost
Marketer	Content + marketing analytics; не изменяет и не открывает orders; без stock/purchase cost/finance

18. Task-oriented UX

- Admin/Manager получают подробную операционную информацию.
- Warehouse получает минимально необходимую информацию.
- Каждый task: Open → данные → Action → Validation → Confirmation → State change → Action Log → Next task.
- На экране одна главная задача и одна основная кнопка.
- Сканирование — preferred; manual input — fallback.
- Warehouse error: простое сообщение. Admin: технические детали в Action Log/technical channel.
- Незавершённую задачу можно открыть позже и продолжить.
- Не создавать отдельное warehouse mobile app на MVP.

19. Notifications

- Admin panel — основная рабочая среда.
- Telegram — notification channel.
- Предпочтительно одна Operations Telegram group для Yandex + VITALO notifications.
- System errors направляются Admin/technical chat.
- Уведомления не должны содержать лишние финансовые или персональные данные.

20. Data model — целевая архитектура

Product — SKU, names RU/UZ, category, brand, manufacturer, country, variant, form, unit, composition, description, usage, contraindications, photos, prices, Yandex commission/logistics.

Purchase — source type, Ozon account/supplier, order number if applicable, purchase date, original total, final actual total, status.

PurchaseLine — product/SKU, ordered/cancelled/received qty, original unit price, allocated final unit cost.

PhysicalUnit — physical ID, product, purchase/purchase line, actual cost, expiry, status, timestamps.

Order — channel, external order number, customer, status, payment status, delivery method, amounts, timestamps, source metadata.

OrderItem — order, product/SKU, quantity, priceAtOrder, assigned PhysicalUnit when assigned.

Reservation — preserve existing VITALO quantity reserve; separate physical assignment concept after payment.

Sale — order, channel, product, physical unit, price, costAtSale, Yandex commission/logistics at sale, advertising allocation if used, profit, soldAt.

Expense — period/date, category, channel, amount, comment, creator.

Return — order/channel, physical unit when known, status, reason, received/accepted time, result RESTOCK/WRITE_OFF.

WriteOff — physical unit, reason, creator, second confirmation, timestamp.

StockMovement — physical unit/product compatibility, movement type, from/to status, reference, actor, timestamp.

Task — type, reference, assignee, status, priority, timestamps.

Employee/Role — reuse existing Telegram-based auth/session model wherever possible.

Dictionaries — Category, Brand, Manufacturer, Country, Form, Unit, ExpenseCategory, Supplier, OzonAccount.

21. Invariants / non-negotiable business rules

- Physical ID unique and never reused.
- One PhysicalUnit cannot belong to two active channel deliveries.
- No double sale.
- No double return acceptance.
- No silent deletion of historical stock/order/purchase/sale events.
- Critical stock transitions must be idempotent.
- Order creation/payment confirmation/stock assignment must be transaction-safe.
- Yandex commission/logistics are snapshot values at sale.
- Historical sale must not recalculate from current Product settings.

- Write-off preserves PhysicalUnit history.
- Purchase cancellation preserves original purchase history.
- Existing VITALO 12h reserve remains functional.

22. Pricing synchronization

- Product stores Yandex price and VITALO price.
- User changes price in internal system first.
- On save ask: «Вы подняли цену в Яндекс Маркете?» and «Вы поменяли цену в VITALO?»
- Yes → mark synchronized. No → mark as requiring external sync.
- Current MVP has no external price API; later this can be replaced by API synchronization.
- Second-stage feature, not MVP: preview of margin/profit impact before changing price.

23. Analytics

- Dashboard: current state — revenue, sales, stock, tasks, problems, expiry alerts.
- Sales: factual sale list.
- Analytics: why result happened.
- Dimensions: date, channel, SKU, supplier, stock, purchase, ad expense.
- Metrics: revenue, units, cost, profit/contribution, margin, average check, top profit SKUs, low-margin SKUs, channel performance, stock, expiry, cash tied up.
- No duplicate pages just for displaying the same underlying data.

24. Expenses and finance

- Finance section is Admin-only.
- Expense has date/period, category, channel, amount, comment.
- User can create custom expense categories.
- Advertising expense is included in analytics.
- Warehouse/Marketer must not see purchase cost or finance unless their role explicitly allows it.

25. Implementation order

- A. Audit and compatibility map
- B. Environment isolation verification
- C. Schema extension / migrations
- D. Core PhysicalUnit + StockMovement + Purchase
- E. Receiving + labels
- F. FEFO + Physical ID assignment
- G. Unified Orders adapter layer
- H. Yandex operational flow
- I. VITALO physical assignment integration
- J. Returns / write-off / corrections
- K. Tasks + warehouse UX
- L. Sales + expense snapshots
- M. Analytics

- N. Telegram notifications / operations group integration if technically safe
- O. Tests / UAT
- P. Production release preparation

26. Testing requirements

- Unit tests for FEFO selection, cost allocation, status transitions and permission checks.
- Integration tests for receiving, order allocation, sale, return, write-off and correction.
- Idempotency tests: repeated callback/action must not duplicate stock movement or sale.
- Concurrency test: two orders cannot reserve/assign the same PhysicalUnit.
- MISSING test: missing FEFO unit triggers exception and next FEFO candidate.
- Return double-acceptance test.
- Purchase discrepancy test: expected < received and expected > received.
- VITALO 12h reserve regression test.
- Existing Mini App checkout regression tests must continue to pass.

27. Acceptance scenarios

- 1. Receive 5 units from Ozon, all with expiry → 5 unique Physical IDs → 5 labels → In Stock.
- 2. Receive expected 2 but actual 1 → one In Stock, one Expected.
- 3. Receive expected 2 but actual 3 → three In Stock + discrepancy.
- 4. Yandex order for one SKU → FEFO selects exact Physical ID → assembly task → Yandex Delivery.
- 5. Selected Physical ID is missing → MISSING + log → next FEFO unit assigned.
- 6. Yandex customer received → manual confirmation → Yandex Sold → Sale created once.
- 7. Yandex return arrives → order number identifies unit → accept → In Stock.
- 8. VITALO order created → existing 12h quantity reserve remains unchanged.
- 9. VITALO payment confirmed → FEFO exact Physical ID → assembly → delivery → sold.
- 10. Write-off requires second confirmation and preserves history.
- 11. Historical Yandex sale remains unchanged when current commission/logistics are edited.

28. Production safety and release gate

- Production release only after test environment passes acceptance scenarios.
- Before migration: verified backup and rollback procedure.
- Verify test and production environment variables and database separation.
- No production data reset.
- Migrations must be backward-aware where required by existing live order flow.
- Deploy in controlled steps and verify health after each step.
- If a migration or feature can affect current VITALO orders, explicitly test the existing order path before release.

29. Claude Code — exact operating instructions

```
You are extending an existing VITALO production-ready MVP.
DO NOT start by rewriting the project.

PHASE 1 — READ-ONLY AUDIT
```

- Inspect the complete repository available in the local Claude Code environment.
- Inspect Mini App, Admin, Backend/API, Prisma schema, migrations, tests, scripts and configuration.
- Trace current order, payment-check, reservation, stock and role flows.
- Inspect local/test and production configuration.
- Determine actual DB separation and infrastructure boundaries.
- Produce a concise CURRENT STATE REPORT.
- Produce a COMPATIBILITY MATRIX:
EXISTS / EXTEND / NEW / CONFLICT / NEEDS DECISION.
- Produce an IMPLEMENTATION PLAN with migration risks.
- Do not modify production.
- Do not run destructive commands.
- Do not invent architecture that has not been observed.

PHASE 2 — IMPLEMENTATION
Only after the audit/plan is clear:
- preserve working Mini App behavior;
- preserve current VITALO 12h pre-payment quantity reservation;
- extend existing entities where appropriate;
- implement PhysicalUnit, FEFO, purchases, receiving, returns, write-offs, tasks and unified operational orders;
- implement Yandex through an adapter boundary so Telegram parser can later be replaced by official API;
- integrate exact Physical ID assignment after VITALO payment confirmation;
- keep Admin as the work environment;
- keep Telegram as notification channel;
- preserve current visual language;
- avoid over-engineering and unnecessary services.

For every schema/API/UI change:
1. explain what existing behavior it touches;
2. implement the smallest safe change;
3. add/update tests;
4. verify migration;
5. verify existing flows still work.

Never silently delete historical data.
Never silently recalculate historical sales.
Never assign the same PhysicalUnit to two active deliveries.
Never make production changes as part of local experimentation.

30. Definition of Done

- Audit report completed.
- Current schema mapped to target model.
- Test/production isolation verified or remediation plan approved.
- Database migrations applied successfully in test.
- Receiving creates Physical IDs and labels.
- FEFO works and handles MISSING.
- Yandex and VITALO use unified physical stock.
- Existing VITALO pre-payment reserve remains intact.
- Sales, returns, write-offs and movements are historically traceable.
- Warehouse task UX works for non-technical family members.
- Roles enforce access.
- Critical flows are idempotent and transaction-safe.
- Tests and UAT scenarios pass.
- Production release plan, backup and rollback documented.

Финальная инструкция Claude: если фактический repository противоречит этому документу, не «исправляй» требование молча. Покажи: (1) текущую реализацию, (2) требование, (3) конфликт, (4) минимальный безопасный способ совместить их. Приоритет — сохранить работающий MVP и постепенно превратить его в Operations-систему.

Документ создан как Этап 6 на основе утверждённых решений Этапов 1–5 и предоставленного обзора текущего VITALO. Он является техническим ТЗ/инструкцией для Claude Code, а не результатом самостоятельного code audit.